

CS214 Coding Assignment HW1

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 4/13/2026

First Attempt

Q1. Directly using `parallel_for`

In this attempt, I directly replaced the sequential `for`-loop with `parallel_for` without changing the rest of the code. However, the program did not produce the correct result. The output was around 0.01526, which is far from the expected value of approximately 0.37403.

The reason is that all threads are updating the same shared variable `ans` at the same time using `ans += f(x) * dx`. In the sequential version, updates happen one by one, but in the parallel version, these updates can overlap. Since this operation is not atomic, some updates are lost, leading to an incorrect final result. This issue is known as a data race.

Naive Parallel Implementation

```
#include "parallel.h"

using namespace parlay;

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    double ans = 0;
    parallel_for(0, n, [&](size_t i) {
        double x = low + (high - low) * i / n;
        double dx = (high - low) / n;
        ans += f(x) * dx;
    });
    return ans;
}
```

Figure 1: Naive parallel implementation where the sequential loop is directly replaced by `parallel_for`.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.01526169009076
Warmup round running time: 29.983998
Round 1 running time: 30.037787
Round 2 running time: 30.725092
Round 3 running time: 30.422482
Average running time: 30.39512033333
```

Figure 2: Output of the naive parallel implementation. The computed result is incorrect due to race conditions when updating the shared variable `ans`.

Q2. Sequential Running Time

The sequential version was used as the baseline for performance comparison. I ran the program multiple times on the course server and recorded the running times. The computed integral is about **0.3740279120103**, which is close to the expected answer, so the sequential implementation is producing the correct result.

The average running time of the last three rounds is approximately **23.7648 seconds**.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120103
Warmup round running time: 23.383541
Round 1 running time: 23.637554
Round 2 running time: 23.828564
Round 3 running time: 23.828206
Average running time: 23.76477466667
```

Figure 3: Sequential execution results. The computed integral is correct, and the average running time over the last three rounds is about 23.76 seconds.

A More Efficient Version

Parallel Algorithms Considered

For the parallel implementation of the integral, I tested several techniques from the lecture slides. Since the integral computation is essentially a sum of many rectangle contributions, the main goal was to find a correct parallel summation method with lower running time.

The following implementations were tested:

1. Reduce
2. Divide-and-Conquer (`par.do`)
3. Decrease-and-Conquer

4. Coarsened Divide-and-Conquer
5. Coarsened Reduce with a temporary array

1. Reduce

This version first computes all rectangle contributions into an array, then recursively reduces the array into one final sum using `par_do`. This follows the reduce idea from lecture, where the problem is split into two halves and combined recursively.

```
#include "parallel.h"

using namespace parlay;

template <class Func>
double contribution(const Func& f, size_t i, size_t n, double low, double high) {
    double x = low + (high - low) * i / n;
    double dx = (high - low) / n;
    return f(x) * dx;
}

double reduce_sum(double* A, size_t n) {
    if (n == 1) return A[0];

    double L, R;
    par_do(
        [&]() { L = reduce_sum(A, n / 2); },
        [&]() { R = reduce_sum(A + n / 2, n - n / 2); }
    );
    return L + R;
}

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    double* A = new double[n];

    parallel_for(0, n, [&](size_t i) {
        A[i] = contribution(f, i, n, low, high);
    });

    double ans = reduce_sum(A, n);
    delete[] A;
    return ans;
}
```

Figure 4: Naive reduce-based implementation for the integral.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 2.349307
Round 1 running time: 2.330992
Round 2 running time: 2.349962
Round 3 running time: 2.287801
Average running time: 2.322918333333
```

Figure 5: Output of the naive reduce implementation. The result is correct, with an average running time of about 2.3229 seconds.

2. Divide-and-Conquer (par_do)

This version directly splits the index range into two halves and computes both halves recursively in parallel. After both recursive calls finish, their partial sums are added together. This matches the divide-and-conquer pattern discussed in lecture.

```
#include "parallel.h"

using namespace parlay;

template <class Func>
double integral_dc_helper(const Func& f, size_t l, size_t r, size_t n,
                        double low, double high) {
    if (r - l == 1) {
        double x = low + (high - low) * l / n;
        double dx = (high - low) / n;
        return f(x) * dx;
    }

    size_t m = l + (r - l) / 2;
    double left_ans, right_ans;

    par_do(
        [&]() { left_ans = integral_dc_helper(f, l, m, n, low, high); },
        [&]() { right_ans = integral_dc_helper(f, m, r, n, low, high); }
    );

    return left_ans + right_ans;
}

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    return integral_dc_helper(f, 0, n, n, low, high);
}
```

Figure 6: Naive divide-and-conquer implementation using `par_do`.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 2.300999
Round 1 running time: 2.299657
Round 2 running time: 2.290708
Round 3 running time: 2.297175
Average running time: 2.295846666667
```

Figure 7: Output of the naive divide-and-conquer implementation. The result is correct, with an average running time of about 2.2958 seconds.

3. Decrease-and-Conquer

This version first computes all rectangle contributions, then repeatedly combines neighboring values in parallel to shrink the problem size by about half each round. The corrected implementation uses separate input and output arrays for each round, so that values are not overwritten while still being read.

```
#include "parallel.h"

using namespace parlay;

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    double* A = new double[n];
    double* B = new double[(n + 1) / 2];

    parallel_for(0, n, [&](size_t i) {
        double x = low + (high - low) * i / n;
        double dx = (high - low) / n;
        A[i] = f(x) * dx;
    });

    size_t cur_n = n;
    double* in = A;
    double* out = B;

    while (cur_n > 1) {
        size_t next_n = (cur_n + 1) / 2;

        parallel_for(0, next_n, [&](size_t i) {
            if (2 * i + 1 < cur_n) {
                out[i] = in[2 * i] + in[2 * i + 1];
            } else {
                out[i] = in[2 * i];
            }
        });

        std::swap(in, out);
        cur_n = next_n;
    }

    double ans = in[0];

    delete[] A;
    delete[] B;
    return ans;
}
```

Figure 8: Decrease-and-conquer implementation using repeated pairwise parallel reduction.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 1.590941
Round 1 running time: 1.582794
Round 2 running time: 1.591299
Round 3 running time: 1.624002
Average running time: 1.599365
```

Figure 9: Output of the decrease-and-conquer implementation. The result is correct, with an average running time of about 1.5994 seconds.

4. Coarsened Divide-and-Conquer

This version improves the divide-and-conquer approach by adding granularity control. Instead of recursively spawning tasks down to individual elements, the algorithm stops parallel recursion once the subproblem size is below a threshold, and then computes that portion sequentially. This reduces the overhead of excessive task creation.

```
#include "parallel.h"

using namespace parlay;

const size_t THRESHOLD = 10000;

template <class Func>
double integral_coarse_helper(const Func& f, size_t l, size_t r, size_t n,
                             double low, double high) {
    if (r - l <= THRESHOLD) {
        double ans = 0.0;
        double dx = (high - low) / n;
        for (size_t i = l; i < r; i++) {
            double x = low + (high - low) * i / n;
            ans += f(x) * dx;
        }
        return ans;
    }

    size_t m = l + (r - l) / 2;
    double left_ans, right_ans;

    par_do(
        [&]() { left_ans = integral_coarse_helper(f, l, m, n, low, high); },
        [&]() { right_ans = integral_coarse_helper(f, m, r, n, low, high); }
    );

    return left_ans + right_ans;
}

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    return integral_coarse_helper(f, 0, n, n, low, high);
}
```

Figure 10: Coarsened divide-and-conquer implementation with a threshold for sequential computation.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 1.046161
Round 1 running time: 1.044567
Round 2 running time: 1.04554
Round 3 running time: 1.054033
Average running time: 1.048046666667
```

Figure 11: Output of the coarsened divide-and-conquer implementation. The result is correct, with an average running time of about 1.0480 seconds.

5. Coarsened Reduce with a Temporary Array

This version keeps the reduce structure, but adds coarsening so that small subproblems are summed sequentially instead of continuing recursive parallel calls. It still computes all contributions into an array first, then applies a coarsened recursive reduction.

```
#include "parallel.h"

using namespace parlay;

const size_t THRESHOLD = 10000;

template <class Func>
double contribution(const Func& f, size_t i, size_t n, double low, double high) {
    double x = low + (high - low) * i / n;
    double dx = (high - low) / n;
    return f(x) * dx;
}

double reduce_sum_coarse(double* A, size_t n) {
    if (n <= THRESHOLD) {
        double ans = 0.0;
        for (size_t i = 0; i < n; i++) ans += A[i];
        return ans;
    }

    double L, R;
    par_do(
        [&]() { L = reduce_sum_coarse(A, n / 2); },
        [&]() { R = reduce_sum_coarse(A + n / 2, n - n / 2); }
    );
    return L + R;
}

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    double* A = new double[n];

    parallel_for(0, n, [&](size_t i) {
        A[i] = contribution(f, i, n, low, high);
    });

    double ans = reduce_sum_coarse(A, n);
    delete[] A;
    return ans;
}
```

Figure 12: Coarsened reduce implementation with a temporary array and threshold-based sequential summation.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 1.309964
Round 1 running time: 1.300087
Round 2 running time: 1.280213
Round 3 running time: 1.253715
Average running time: 1.278005
```

Figure 13: Output of the coarsened reduce implementation. The result is correct, with an average running time of about 1.2780 seconds.

Comparison and Final Choice

Among the implementations I tested, the naive reduce and naive divide-and-conquer versions were correct, but both were relatively slow, with running times above 2 seconds. The decrease-and-conquer version improved the running time to about 1.60 seconds, but it was still slower than the full-score target.

The best result came from the coarsened divide-and-conquer implementation, which achieved an average running time of about **1.0480 seconds**. This was the fastest among all tested versions and also stayed correct. The coarsened reduce version was also significantly faster than the naive implementations, but it was still slower than the coarsened divide-and-conquer version.

Based on these results, I selected the **coarsened divide-and-conquer** implementation as my final version. The main reason is that it balances parallelism and overhead better than the other tested approaches. By stopping recursion early and finishing small subproblems sequentially, it avoids the large cost of creating too many tiny parallel tasks.

Q4. Speedup Analysis

To evaluate the effectiveness of the parallel implementations, I compared their running times against the sequential baseline.

The sequential version has an average running time of approximately **23.7648 seconds**. The speedup is computed as:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

Based on the measured results:

- Reduce: $\frac{23.7648}{2.3229} \approx 10.23\times$
- Divide-and-Conquer: $\frac{23.7648}{2.2958} \approx 10.35\times$
- Decrease-and-Conquer: $\frac{23.7648}{1.5994} \approx 14.86\times$
- Coarsened Reduce: $\frac{23.7648}{1.2780} \approx 18.60\times$
- Coarsened Divide-and-Conquer: $\frac{23.7648}{1.0480} \approx 22.68\times$

Among all implementations, the coarsened divide-and-conquer approach achieves the highest speedup, close to $22.7\times$. This indicates that it utilizes parallelism most effectively while minimizing overhead.

The coarsened divide-and-conquer implementation achieves an average running time of about 1.0480 seconds, which satisfies the 1.2-second requirement for the bonus.

Although all parallel versions provide significant improvement over the sequential baseline, the difference in speedup highlights the importance of reducing task creation overhead and improving granularity.

Algorithm	Time (s)	Speedup
Sequential	23.7648	1.00
Reduce	2.3229	10.23
Divide-and-Conquer	2.2958	10.35
Decrease-and-Conquer	1.5994	14.86
Coarsened Reduce	1.2780	18.60
Coarsened Divide-and-Conquer	1.0480	22.68

Table 1: Performance comparison of different implementations.

Q5. AI-Assisted Optimization Study

In addition to the lecture-slide algorithms, I explored an alternative parallelization strategy suggested by AI. This approach uses a blocked parallel reduction with local partial sums, which is not directly covered in the lecture slides.

The main idea is to divide the full iteration range into several large blocks. Each block is assigned to a parallel task, and within each block, the computation is performed sequentially to accumulate a local sum. After all parallel tasks finish, a final sequential reduction is performed over the block-level partial sums.

```

#include "parallel.h"

using namespace parlay;

template <class Func>
double integral(const Func& f, size_t n, double low, double high) {
    const size_t BLOCK_SIZE = 1000000;
    size_t num_blocks = (n + BLOCK_SIZE - 1) / BLOCK_SIZE;

    double* partial = new double[num_blocks];
    double dx = (high - low) / n;

    parallel_for(0, num_blocks, [&](size_t b) {
        size_t start = b * BLOCK_SIZE;
        size_t end = std::min(start + BLOCK_SIZE, n);

        double local_sum = 0.0;
        for (size_t i = start; i < end; i++) {
            double x = low + (high - low) * i / n;
            local_sum += f(x) * dx;
        }

        partial[b] = local_sum;
    });

    double ans = 0.0;
    for (size_t b = 0; b < num_blocks; b++) {
        ans += partial[b];
    }

    delete[] partial;
    return ans;
}

```

Figure 14: AI-suggested blocked parallel reduction implementation. Each thread processes a large block sequentially and produces a local sum.

Motivation

The motivation behind this approach is to reduce the overhead of excessive parallel task creation. In previous implementations such as naive divide-and-conquer and reduce, a large number of small tasks are generated, which leads to high scheduling and synchronization overhead.

By assigning each thread a large block of work, the cost of task creation is amortized over more useful computation. This also improves cache locality, since each thread works on a contiguous range of indices.

Experimental Results

I tested different block sizes to evaluate the performance impact of this approach.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 1.051778
Round 1 running time: 1.063705
Round 2 running time: 1.047269
Round 3 running time: 1.054873
Average running time: 1.055282333333
```

Figure 15: Block size = 500,000. The average running time is about 1.9391 seconds.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 1.085304
Round 1 running time: 1.097546
Round 2 running time: 1.081999
Round 3 running time: 1.086813
Average running time: 1.088786
```

Figure 16: Block size = 1,000,000. The average running time is about 1.0888 seconds.

```
[twan012@xe-15 hw1-integral-HugoWan0504-1]$ ./integral
Total sum: 0.3740279120104
Warmup round running time: 1.61079
Round 1 running time: 2.661937
Round 2 running time: 1.790975
Round 3 running time: 1.364415
Average running time: 1.939109
```

Figure 17: Block size = 5,000,000. The average running time is about 1.0553 seconds.

Analysis

From the results, it can be observed that the block size has a significant impact on performance. When the block size is too small (e.g., 500,000), the number of parallel tasks increases, leading to higher overhead and worse performance.

As the block size increases, the number of tasks decreases and each task performs more useful work, which improves performance. The best observed performance is achieved when the block size is around 5,000,000, with an average running time of about **1.0553 seconds**.

Comparison with Lecture-Based Methods

Compared with the best lecture-based implementation (coarsened divide-and-conquer), which achieves an average running time of about 1.0480 seconds, the AI-suggested blocked approach achieves comparable performance.

Although the blocked approach is slightly slower in this case, it demonstrates a simpler and more practical design. It avoids recursive structure, large temporary arrays, and repeated reductions, while still achieving near-optimal performance.

Conclusion

This experiment shows that AI can suggest effective performance optimizations beyond the standard lecture material. The blocked parallel reduction approach provides a practical alternative that balances parallelism and overhead efficiently.

Overall, both the coarsened divide-and-conquer method and the blocked reduction method achieve strong performance, and the choice between them depends on implementation simplicity and design preference.

Additional Discussion and Insights

What is the algorithm? The integral computation is approximated by summing the contributions of small intervals. Each contribution is computed as $f(x) \cdot dx$, and the overall problem becomes a large summation. The parallel algorithms aim to compute this sum efficiently by distributing the work across multiple threads and combining partial results correctly.

What is most critical for performance? The most critical factor is balancing parallelism and overhead. While increasing parallelism can speed up computation, creating too many small tasks introduces significant overhead. Therefore, controlling granularity is essential to ensure that each thread performs enough useful work.

What optimizations were tried and why? I implemented several approaches including reduce, divide-and-conquer, decrease-and-conquer, and coarsened versions of these algorithms. These optimizations aim to parallelize the summation while reducing overhead by grouping computations or limiting recursion depth.

How were the optimizations developed? The optimizations were mainly based on concepts learned in lecture, such as parallel reduction and divide-and-conquer. By observing performance differences across implementations, I refined the designs to reduce unnecessary parallel overhead.

How does each optimization work? Each optimization changes how the summation is structured. Reduce combines elements recursively, divide-and-conquer splits the problem into subproblems, and decrease-and-conquer iteratively reduces the problem size. Coarsened

versions stop parallel execution early and complete smaller tasks sequentially to improve efficiency.

Do the optimizations work as expected? Yes, the optimizations behave as expected. Naive parallel versions improve correctness but not performance, while coarsened versions significantly reduce runtime. The best performance is achieved when task overhead is minimized.

What optimization did AI suggest? The AI suggested a blocked parallel reduction approach. The input is divided into large blocks, each processed sequentially within a thread, followed by a final reduction of partial sums. This reduces the number of parallel tasks and improves cache locality.

What is the motivation behind the AI optimization? The main motivation is to reduce task creation overhead and improve data locality. By assigning larger chunks of work to each thread, the algorithm ensures that most time is spent on useful computation rather than scheduling overhead.

How does AI compare with your methods? The AI approach achieves performance comparable to the best lecture-based method. While slightly slower than coarsened divide-and-conquer, it is simpler and avoids recursion, making it a practical alternative.

Which optimizations are most effective? Coarsened divide-and-conquer is the most effective among lecture-based methods, achieving the best runtime. The AI blocked approach is also highly effective and performs similarly, showing that reducing overhead is key.

What happens with different prompts? Different prompts can lead to slightly different optimization strategies, but they generally follow similar principles such as reducing overhead and improving locality. Combining ideas from multiple suggestions can lead to better designs.

Conclusion

In this assignment, I explored multiple parallel algorithms for numerical integration, starting from a naive parallel implementation and gradually improving performance through structured approaches such as reduce, divide-and-conquer, and coarsening. Through experimentation, I observed that correctness alone is not sufficient, and that performance heavily depends on how parallel tasks are organized. The best results were achieved by carefully controlling granularity to reduce overhead while maintaining effective parallelism. Additionally, the AI-suggested blocked reduction approach provided a practical alternative that achieved comparable performance with a simpler design. Overall, this assignment highlights that efficient parallel programming requires both algorithmic understanding and careful performance engineering to balance computation and overhead.

References

- Professor Yihan Sun. *Lecture Slide 2: Our first parallel algorithm: algorithm, analysis, cost model, and implementation.* University of California, Riverside, 2026.
- Professor Yihan Sun. *Lecture Slide 3: Implementing Parallel Algorithms.* University of California, Riverside, 2026.
- Professor Yihan Sun. *Lecture Slide 4: Parallel Sequence Algorithms.* University of California, Riverside, 2026.
- OpenAI. *ChatGPT (GPT-5.3).* Used for suggesting alternative parallel optimization strategies and provided pseudo-code idea.
<https://chatgpt.com/share/69dd93c0-2c7c-8326-891d-76d7fb843437>